

1. Introduction

The fully-connected neural network has had profound implications on several industries, leveraging large amounts of data to make useful predictions and decisions based on learned trends from the data. Applications for this technique include pattern classification, time-series prediction, and decision making in games. However, how exactly does this inference model work and what has shaped it to become such a transformative concept?

1.1 History

To understand the progression of the neural network, one has to first grasp the underlying inspiration for this idea: biological neural networks found in the brain. Hierarchically ordered in the brain, neurons (Figure 1) can be simplified down to 4 key parts/functions: the dendrites, which collect incoming information, the soma or cell body, which integrates the information from the dendrites and does some processing on it, the axon, which propagates the input to the output, and the axon terminals, which transmit the neuron output to another neuron's input.

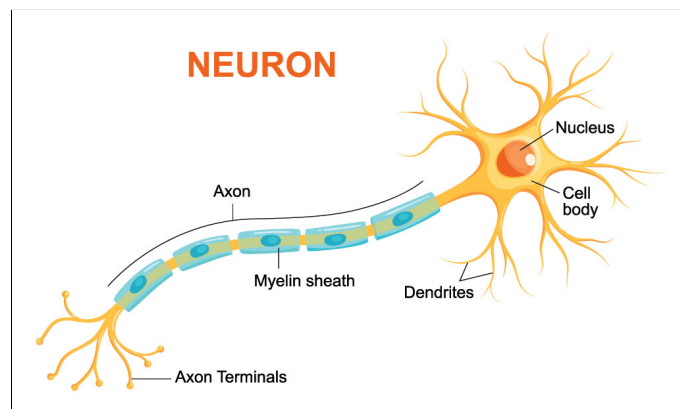


Figure 1: Model of a biological neuron in the brain¹

Knowing this, we can appreciate the model developed by McCulloch and Pitts in 1943, the MCP neuron model, which presented a succinct, computationally feasible model that could perform these simple tasks. As seen in Figure 2, similar to the biological model in Figure 1, we observe the inputs x_1, x_2, x_3 (dendrites) being integrated by the function g , upon which some processing is done to produce an output (soma/axon)³. Lastly, this output is propagated to some output (axon terminals). This representation of the neuron allowed for the model to perform logical operations, all depending on the choice of f .³

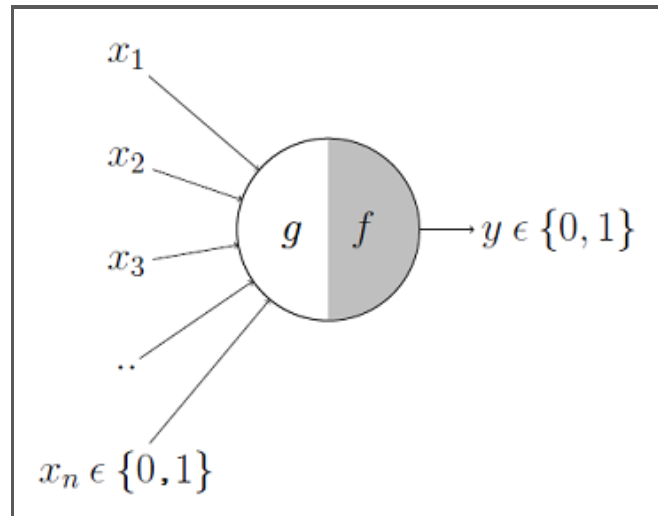


Figure 2: MCP Neuron Model

Still, this idea only began to emulate the functionality of a neural network, as the concept of a network of neurons had not been fully considered. Over the next decade, these ideas developed further, leading up to Frank Rosenblatt's 1958 paper, *Design of an Intelligent Automaton*⁵. As seen in Figure 3, the hierarchical, layered structure of biological neural networks was computationally represented using a perceptron. These ideas flourished over the next decade until Marvin Minsky and Seymour Papert's book in 1969, *Perceptrons*, criticizing the technique over its inability to solve the XOR problem (due to the single-layer considered)⁸. These theoretical setbacks, in addition to lack of computational resources, led to the first AI winter spanning 1974-80².

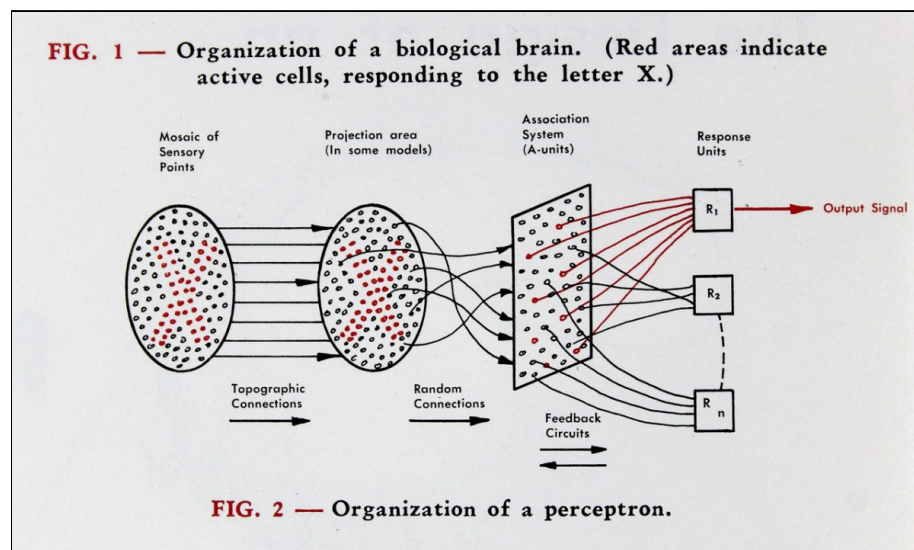


Figure 3: Rosenblatt's multi-layer perceptron⁴

While the ideas surrounding the structure of a neural network had been explored along with the computational structure of individual neurons, the algorithm for learning still left open questions. In 1982, Paul Werbos proposed the backpropagation algorithm for propagating errors in neural networks as a method for network training. A few years later, Geoff Hinton, David Rumelhart and Ronald Williams from the PDP research group practically demonstrated the implementation of backpropagation in neural networks in *Learning representations by back-propagating errors* (1986)⁸.

Following a second AI winter, resulting from overblown expectations on the capabilities of AI systems at the time, practical implementations began to catch up with the theory. Specifically, in 2006, Hinton's group at University of Toronto developed Deep Belief Networks, which began stacking layers into a deeper network, while training using backpropagation on large datasets. Finally, with the rapid adoption of parallel processing power of GPUs for deep networks (advocated by Andrew Ng in 2008), we approach the current status quo, where the theoretical ideas constructed by early researchers have been vindicated by the practical applications of today.

1.2 Motivation

Given the popularity of neural networks in modern-day machine learning applications, we were inspired to create our own neural network library. Additionally, in an effort to understand the mathematical underpinnings and algorithm for this popular machine learning technique, we wanted the experience of building our own neural network library from scratch.

From another standpoint, there is still much to understand behind the theory of neural networks, so this project also gave us an opportunity to empirically investigate and form our own conclusions about some interesting phenomena.

Along the way, we also hoped to gain experience with the popular mathematical Python library, NumPy, as well as practice writing well-abstracted, object-oriented code. Finally, we strived to gain experience using popular machine learning libraries such as PyTorch that are heavily used in both academia and industry.

2. Methods

In this section we demonstrate the algorithm used to train and test the neural network we implemented. We then review how we built our own neural network library based on these mathematical concepts.

2.1 Algorithm

In order to train the neural network, the parameters of the network (weights and biases) were first randomly initialized. The following algorithms shown below were then iterated upon for each training sample for each epoch.

Algorithm 1 Forward Propagation for Weight Update

Ensure: $N = \#$ of layers
Ensure: $\mathbf{x}_1 \leftarrow \text{Input}$
for j in $[2, \dots, N]$ **do**
 if $\text{Type}(j^{\text{th}} \text{ Layer}) = \text{"Fully Connected"}$ **then**
 $\mathbf{x}_j \leftarrow \mathbf{x}_{j-1} \mathbf{W}_{j-1,j} + \mathbf{b}_j$
 else
 $\mathbf{x}_j \leftarrow \sigma(\mathbf{x}_j)$
 end if
end for

Algorithm 2 Backpropagation with Gradient Descent for Loss Minimization

Ensure: $\alpha \leftarrow$ learning rate
Ensure: $\mathbf{e}_N \leftarrow \nabla_{\mathbf{x}_N} \mathcal{L}$ where $\mathcal{L} = \text{Training Error}$
for j in $[N-1, \dots, 1]$ **do**
 if $\text{Type}(j^{\text{th}} \text{ Layer}) = \text{"Fully Connected"}$ **then**
 $\mathbf{e}_j \leftarrow \mathbf{e}_{j+1} \mathbf{W}_{j,j+1}$
 $\mathbf{W}_{j,j+1} \leftarrow \mathbf{W}_{j,j+1} - \alpha * \mathbf{x}_j^T \mathbf{e}_{j+1}$
 $\mathbf{b}_j \leftarrow \mathbf{b}_{j+1} - \alpha * \mathbf{e}_{j+1}$
 else
 $\mathbf{e}_j \leftarrow \sigma'(\mathbf{x}_j) \odot \mathbf{e}_{j+1}$
 end if
end for

It is important to note that though the forward propagation algorithm is used for classification of an input vector for training, validation and test samples, backpropagation is only performed on the training data samples (parameters only ‘learn’ the training data).

Moreover, it is also observed that there are three unspecified parameters: α , $\mathcal{L}$, and σ . Each of these three parameters are tunable and subject to the user’s discretion. Throughout this study, a learning rate of 0.1, the mean-squared error (MSE) loss and the hyperbolic tangent function were used respectively.

2.2 Python Library

The most fundamental element of our library is the abstract `Layer` class. This class has input and output fields as well as forward propagation and backpropagation methods. The rest of our library was built on top of this layer object including the `Fully-Connected Layer` and `Activation Layer` classes which directly inherited from the `Layer` class. Our `Network` class houses most of the functionality of a neural network including a list of its layers and `train` and `predict` methods. We also incorporated a number of tunable hyperparameters for training the network including a learning rate and number of epochs.

Initially, we worked with the CIFAR-10 dataset, but decided to switch to the MNIST dataset for efficiency and ease of use. This way, we did not have to work with large images with multiple channels.

3. Results

In this section we will discuss our results concerning three main research questions

1. What is the effect of changing the number of epochs?
2. What is the effect of changing the number of training samples?
3. How does this neural network implementation compare to that of a standard machine learning library such as PyTorch?

3.1 Effect of Number of Epochs

To answer the first question about the effect of changing the number of epochs during training, we built two simple neural networks using our library. The first neural network was a “shallow” network that consisted of one hidden layer (three layers total). This was implemented in code as follows:

```
network = Network(mse, mse_prime)
network.add_layer(FCLayer(28*28, 100))
network.add_layer(ActivationLayer('tan'))
network.add_layer(FCLayer(100, 10))
```

We trained this network on 5,000 images and tested it with 1,000 images. All parameters were kept the same except for the number of epochs, which we varied between 10, 30, and 100. Graphs of epoch versus corresponding training loss can be seen in Figure 4 for two of the three tests performed with this setup.

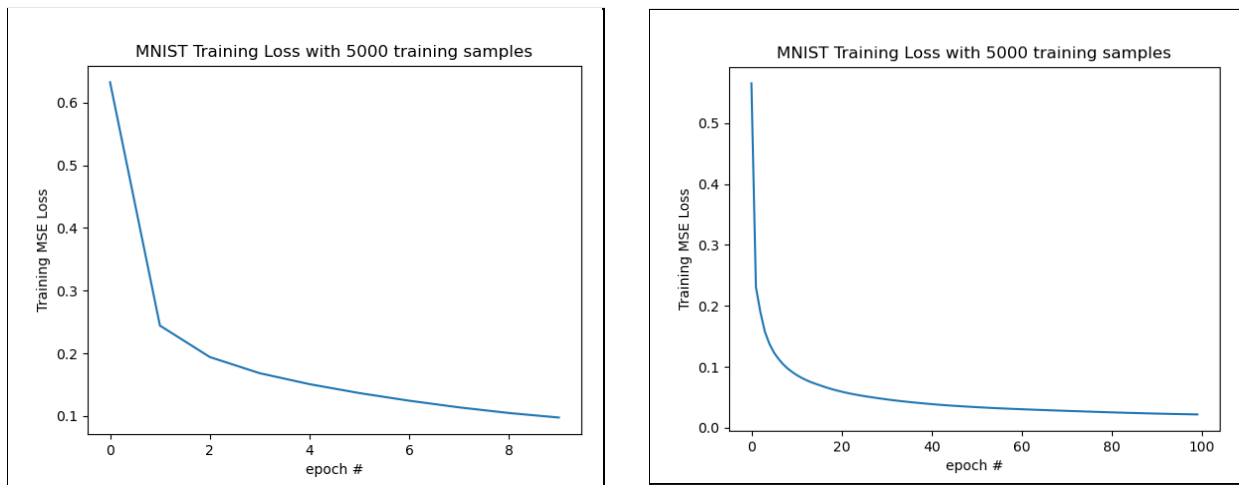


Figure 4: Loss over 10 (Left) and 100 (Right) epochs for shallow network

We then ran the same tests again, this time on a “deep” network with three hidden layers (five layers total). This was implemented in code as follows:

```

network = Network(mse, mse_prime)
network.add_layer(FCLayer(28*28, 28*7))
network.add_layer(ActivationLayer('tan'))
network.add_layer(FCLayer(28*7, 14*7))
network.add_layer(ActivationLayer('tan'))
network.add_layer(FCLayer(14*7, 7*7))
network.add_layer(ActivationLayer('tan'))
network.add_layer(FCLayer(7*7, 10))

```

Graphs of epoch versus corresponding training loss can be seen in Figure 5 for two of the three tests performed with this setup.

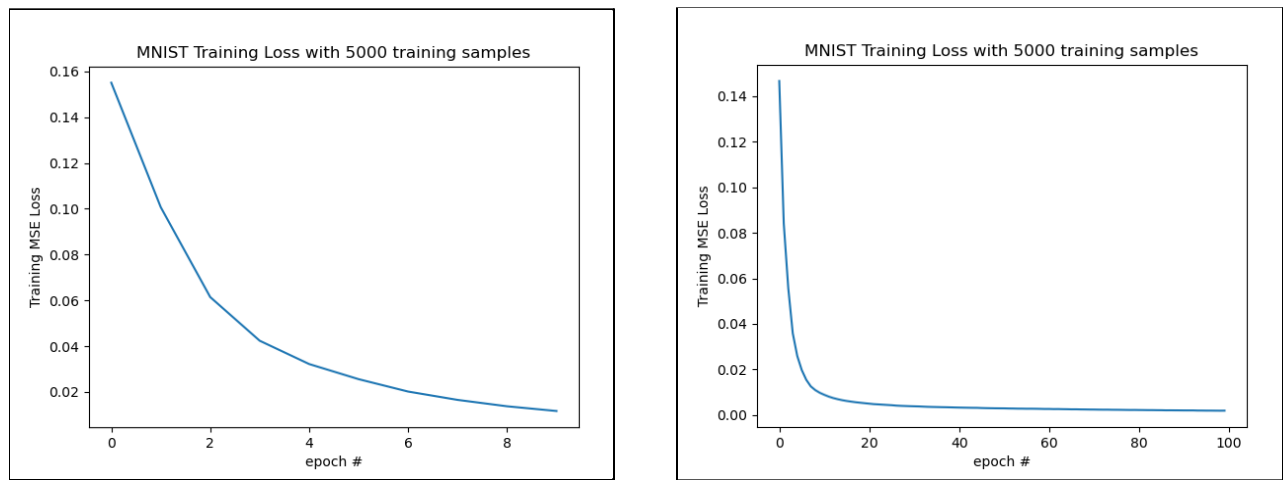


Figure 5: Loss over 10 (Left) and 100 (Right) epochs for deep network.

We can see the general trend that training loss decreases as the number of epochs increases. The resulting train and test accuracies for each of these six tests can be seen in Table 1.

Epochs	Train Accuracy (Shallow)	Train Accuracy (Deep)	Test Accuracy (Shallow)	Test Accuracy (Deep)
10	76.92%	96.60%	71.30%	87.60%
30	91.20%	98.74%	80.00%	87.80%
100	96.34%	99.16%	83.80%	86.50%

Table 1: Train and test accuracies for epoch variation tests

One observation we can make from this data is that increasing the number of epochs increases both the train and test accuracies. This aligns with our belief about how a neural network should function. Another observation we can make is that increasing the number of epochs has a greater effect on the shallow network compared to the deep network. One explanation for this is that the deep network has more learnable parameters, and thus more learning can occur during each training epoch. This means that the shallow network needs more epochs to achieve the same level of training as the deep neural network.

3.2 Effect of Training Data Size

Using the deep network architecture discussed in section 3.1, the number of training samples was then varied over a wide range of values to observe the effect of increased training examples on classification accuracy and loss. Throughout this experiment, each network was trained over 50 epochs and tested on the entire MNIST testing dataset (10000 test samples).

We hypothesized that as the network was presented with more training examples, the generalization capability would increase, thus increasing the test accuracy. However, we also desired a measure for determining whether the model would overfit to the training data apart from classification accuracy. Thus, the loss was also measured on a validation set (15% of the original training dataset) while the remaining data was used for training.

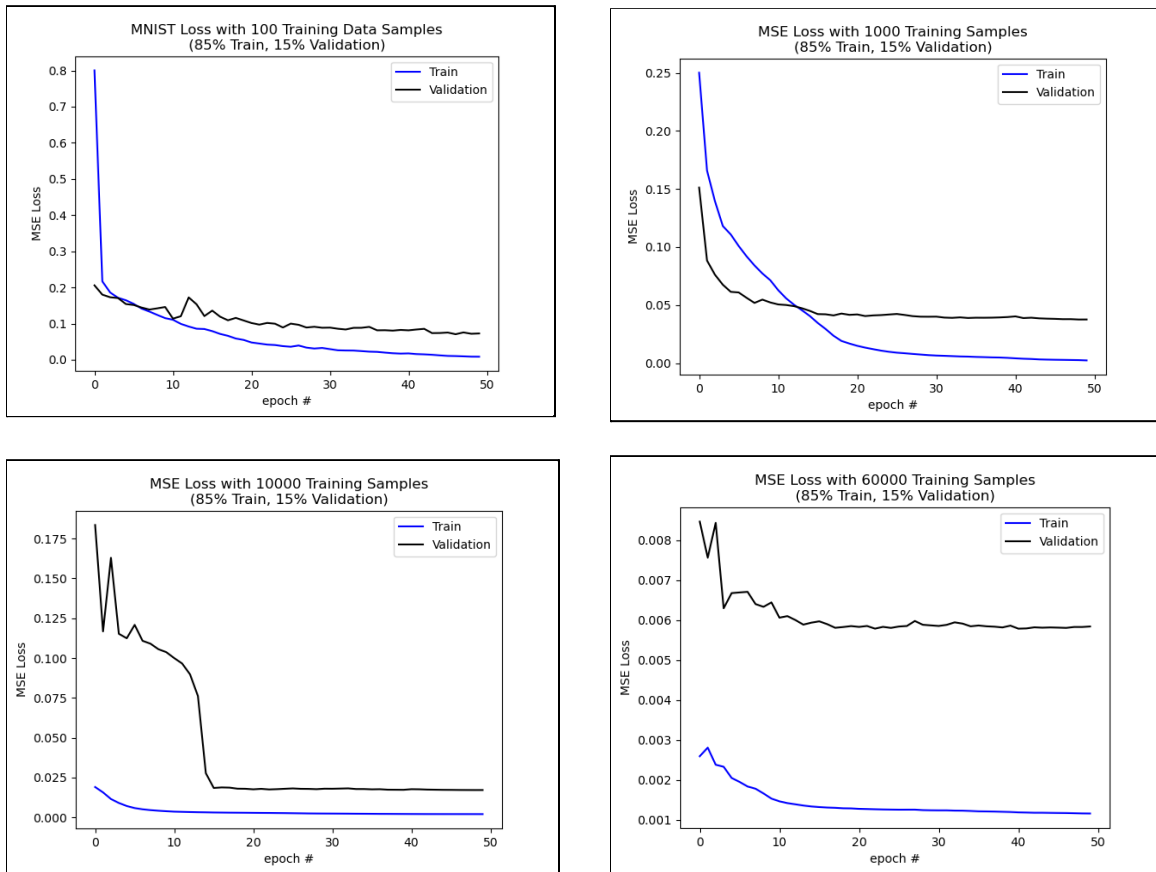


Figure 6: Effect of varying training dataset size on training/validation loss (split 85:15)

The experimental results can be seen in Figure 6, where we observe a common trend of fast-decaying training loss as well as decreasing validation loss. From Figure 6a, despite training on a deep network using only 100 training examples, it is interesting to note that the loss on the validation set (which the network was not trained on) still decreases steadily before converging to just below 0.1. However, it appears that as the number of training samples grows, the training loss converges to a value closer and closer to 0 while the validation loss converges to a similarly lower value, a constant value above the final training loss. This is likely due to the heightened ability of the network to learn more general representations of each type of image, allowing better tuning of the parameters.

Meanwhile, overfitting does not seem to occur over 50 epochs, though the network appears to tune to the training data as the validation loss levels off after around 20 epochs in each case.

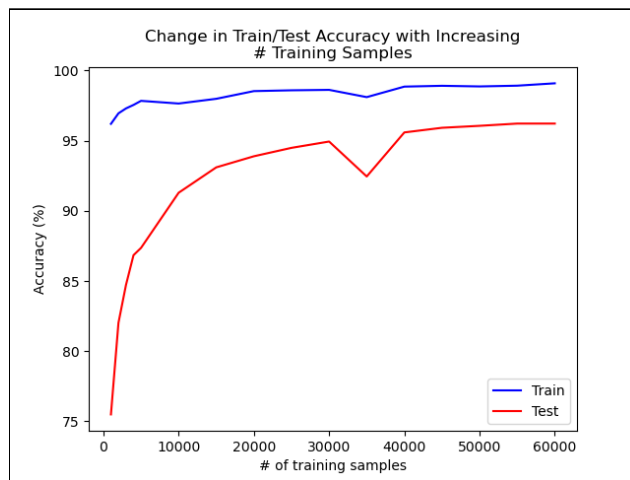


Figure 7: Train and Test accuracy with 10000 test samples, trained over 50 epochs

For each training dataset configuration, the training and test classification accuracy was also measured (Figure 7). As the number of training examples is increased (keeping in mind 15% was used for validation in each case), the training accuracy approaches nearly 98% while the testing accuracy drastically improves from 75% to 95%. Worthy of note, the training accuracy remains nearly the same throughout this variation, which matches expectations due to the network already having learned the training examples. On the other hand, the testing accuracy greatly improves with initial increases in training examples (e.g. > 15% increase from 100 to 10000) while only slightly with larger changes in training examples (e.g. < 5% increase from 10000 to 60000). This empirically observed phenomenon describes the necessity of sufficient training data for proper generalization, while also illustrating the redundancy of training examples needed for marginal improvements in learned representations.

Overall, these results confirm the hypothesis laid out at the beginning of this experiment, albeit with some interesting observations.

3.3 PyTorch Comparison

To test how our neural network implementation compared to a standard machine learning library's implementation (PyTorch in this case), we built neural networks equivalent to our shallow and deep networks. We then trained and tested all four networks on the full MNIST dataset (60,000 training images, 10,000 testing images). The testing accuracies for all four of these networks can be seen in Table 2.

	Ours	PyTorch
Shallow	92.40%	95%
Deep	96.45%	97%

Table 2: Test accuracies for shallow and deep networks for two implementations

The most important observation we can make from this data is that our implementation and the PyTorch implementation perform roughly equivalently. This helps verify that our network was implemented correctly. Another observation we can make is that the deep networks consistently perform better than the shallow networks for both implementations, which also matches expectations.

The PyTorch network also consistently performs slightly better than ours. One possible reason for this is that we use gradient descent to optimize our network while PyTorch uses mini-batch stochastic gradient descent. In our implementation of gradient descent, we update all of the parameters of the network for every single training image, in order. This is performed for every training epoch. Another possible reason is that PyTorch performs some additional normalization of the data, which might allow the network to learn the fundamental representations of the images more quickly. Lastly, one other difference is that PyTorch uses cross-entropy loss (as opposed to mean squared error in our network), which has been proven to converge faster in classification problems⁹. This is due to the larger decision boundaries inherent in such problems which cross-entropy loss takes further advantage of compared to MSE loss. .

4. Conclusion

In this section we present a summary of our main results as well as discuss some possible future improvements of this neural network library.

4.1 Discussion

Research questions one and two verified the validity and intuition behind our implementation of a neural network. This confirmed that both increasing the number of epochs and the number of training samples of our neural network enhances its performance. Research question three proved

that our implementation closely matches that of PyTorch in terms of results when training and testing on the MNIST database. One main difference between our implementation and PyTorch was the training time. Our network often took about 5x longer to train compared to training a similar network with PyTorch. One possible explanation for this is PyTorch's inclusion of momentum when calculating the gradient vectors.

4.2 Future Study

There are a few ways in which our library could be improved, as well as several experiments that could be run. For instance, the ReLU activation function could be used as the nonlinear activation layer instead of the hyperbolic tangent, which has also been shown to remediate the vanishing gradient problem. Moreover, as discussed towards the end of Section 4.1, the addition of cross-entropy loss could prove to be very valuable in classification problems. We have been working on these additional improvements and look forward to the interesting results.

Another experiment that could be run would be to observe the effects of techniques such as weight decay, early stopping, and batch normalization. Moreover, the library could be tested on additional, larger databases including CIFAR-10 to obtain a better understanding of the limitations of our network's architecture.

A couple of interesting research questions that we have not investigated yet include:

1. Can the double descent phenomena be observed on this network by introducing label noise and increasing the number of parameters in the network?
2. Does depth of the network matter? Specifically, by keeping the number of network parameters constant while varying the number of hidden layers, will the train/test accuracy vary?

5. References

- [1]
<https://www.istockphoto.com/vector/illustration-of-neuron-anatomy-vector-infographic-gm1153647071-313399023>
- [2]
<https://www.historyofdatascience.com/ai-winter-the-highs-and-lows-of-artificial-intelligence/>
- [3]
<https://medium.com/@datamonsters/artificial-neural-networks-in-natural-language-processing-bcf62aa9151a>
- [4]
<https://towardsdatascience.com/mcculloch-pitts-model-5fdf65ac5dd1>
- [5]
<https://news.cornell.edu/stories/2019/09/professors-perceptron-paved-way-ai-60-years-too-soon>
- [6]
<https://towardsdatascience.com/math-neural-network-from-scratch-in-python-d6da9f29ce65>
- [7]
<https://nextjournal.com/gkoehler/pytorch-mnist>
- [8]
https://machinelearningknowledge.ai/brief-history-of-deep-learning/#Frank_Rosenblatt_creates_Perceptron
- [9]
https://susanqq.github.io/tmp_post/2017-09-05-crossentropyvsmses/

6. Appendix

For more details, please view our library here: <https://github.com/saakethmm/custom-nn>